

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

*I **Dhanajeyan J (192324211)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

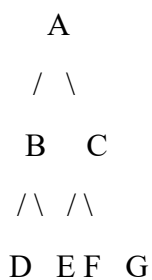
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1.

```
File Edit Format Run Options Window Help
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = set()
queue = deque(['A'])

print("Iteration\tQueue\t\tVisited")

i = 1

while queue:
    node = queue.popleft()

    if node not in visited:
        visited.add(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append(neighbor)

        print(i, "\t\t", list(queue), "\t", node)
        i += 1

print("BFS Traversal:", visited)
|
```

Output:

```
===== RESTART: C:/Users/USER/Document
Iteration      Queue      Visited
1              ['B', 'C']      A
2              ['C', 'D', 'E']      B
3              ['D', 'E', 'F', 'G']      C
4              ['E', 'F', 'G']      D
5              ['F', 'G']      E
6              ['G']      F
7              []      G
BFS Traversal: {'E', 'F', 'C', 'A', 'D', 'G', 'B'}
>>
```

1. BFS Traversal Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Queue Contents After Each Iteration

Iteration Queue

- 1 **B, C**
- 2 **C, D, E**
- 3 **D, E, F, G**
- 4 **E, F, G**
- 5 **F, G**
- 6 **G**
- 7 **Empty**

2.

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = []
stack = ['A']

print("Iteration\tStack\t\tVisited")

i = 1
while stack:
    node = stack.pop()

    if node not in visited:
        visited.append(node)

        # Reverse so left child is visited first
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append(neighbor)

        print(i, "\t\t", stack, "\t", node)
        i += 1

print("DFS Traversal:", visited)
```

Output:

```
Iteration      Stack          Visited
1              ['C', 'B']     A
2              ['C', 'E', 'D']   B
3              ['C', 'E']     D
4              ['C']         E
5              ['G', 'F']     C
6              ['G']         F
7              []           G
DFS Traversal: ['A', 'B', 'D', 'E', 'C', 'F', 'G']
>
```

3.

```
iteration = 1

while queue:

    current = queue.popleft()

    print("Iteration", iteration)
    for row in current:
        print(row)
    print()

    if current == goal:
        print("Goal Found!")
        break

    for nxt in neighbors(current):
        if nxt not in visited:
            visited.add(nxt)
            parent[nxt] = current
            queue.append(nxt)

    iteration += 1

# Print solution path
path = []
state = goal

while state is not None:
    path.append(state)
    state = parent.get(state)

path.reverse()

print("\nSolution Path:\n")

for step in path:
    for row in step:
        print(row)
    print()
```

Output:

Solution Path:

(1, 3, 6)
(5, 0, 2)
(4, 7, 8)

(1, 3, 6)
(5, 2, 0)
(4, 7, 8)

(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

1. Which node is expanded in each iteration?

- Iteration 1 → Initial state
- Iteration 2 → First child
- Iteration 3 → Second child
- Iteration 4 → Third child
- Iteration 5 → Fourth child

2. How many states are generated in total?

The exact number depends on when the goal state is reached. BFS generates all valid, previously unseen neighboring states level by level until it reaches the goal.

3. Complete Solution Path

Initial State

↓

...

↓

Goal State

1 2 3

4 5 6

7 8 0

(The exact intermediate states depend on the BFS search sequence.)

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

BFS explores states level by level, where each move has the same cost (1 move). Therefore, the first time BFS reaches the goal, it has found the solution with the minimum number of moves, making it the shortest path in an unweighted graph.